

Teoria Współbieżności

Laboratorium 9

Przetwarzanie asynchroniczne (wstęp do Node.js)

Kacper Bieniasz

2 grudnia 2024

1 Dane techniczne sprzętu

Obliczenia zostały wykonane na komputerze o następującej specyfikacji:

- Procesor: AMD Ryzen 7 5800U 8 rdzeeni, 16 wątków
- Pamięć RAM: 16 GB DDR4 3200 MHz (2×8GB)
- System operacyjny: Windows 11 Home x64

2 Przetwarzanie asynchroniczne w Node.js

W celu zaobserwowania działania asynchronicznego kilkukrotnie uruchamiam program (1). Możemy zauważyć, że kolejność wypisywanych cyfr nie jest od siebie zależna. Każde wywołanie jest oddzielne i otrzymuję swój losowy czas opóźnienia.

```
1 function printAsync(s, cb) {  
2   var delay = Math.floor((Math.random()*1000)+500);  
3   setTimeout(function() {  
4     console.log(s);  
5     if (cb) cb();  
6   }, delay);  
7 }  
8 printAsync("1");  
9 printAsync("2");  
10 printAsync("3");  
11 console.log('done!');
```

Kod w JavaScript 1: Program działający asynchronicznie

Chcąc wymusić sekwencyjne zachowanie możemy to zrobić w następujący sposób, jako argument *cb* przekazać kolejne wywołanie funkcji z inną cyfrą. Takie podejście prezentuje kod (2). Program z taką modyfikacją działa poprawnie, jednak nie jest to elegancki sposób na radzenie sobie z tego typu problemami. Takie rozwiązanie zmniejsza czytelność kodu.

```

1 function printAsync(s, cb) {
2     var delay = Math.floor((Math.random()*1000)+500);
3     setTimeout(function() {
4         console.log(s);
5         if (cb) cb();
6     }, delay);
7 }
8 printAsync("1", function() {
9     printAsync("2", function() {
10         printAsync("3", function() {
11             console.log('done!');
12         });
13     });
14 });

```

Kod w JavaScript 2: Wykorzystanie *callback*

Drugim sposobem na zapewnienie sekwencyjnego wykonania jest użycie funkcji **execute()**, która przyjmuje listę funkcji do wykonania, które jako argument przyjmują *callback*. Funkcja **execute()** zapewnia sekwencyjność poprzez ustalanie zależności między otrzymanymi zadaniami. Realizuję to co kod (2), ale nie pogarsza czytelności. Kod tego sposób przedstawia blok (3).

```

1 function printAsync(s, cb) {
2     var delay = Math.floor((Math.random() * 1000) + 500);
3     setTimeout(function () {
4         console.log(s);
5         if (cb) cb();
6     }, delay);
7 }
8 function execute(tasks) {
9     function iterate(index) {
10         if (index === tasks.length) {
11             return;
12         }
13         const task = tasks[index];
14         task(() => iterate(index + 1));
15     }
16     return iterate(0);
17 }
18 function task1(cb) {
19     printAsync("1", function () {
20         cb();
21     });
22 }
23 function task2(cb) {
24     printAsync("2", function () {
25         cb()
26     });
27 }
28 function task3(cb) {
29     printAsync("3", function () {
30         cb()
31     });
32 }
33 execute([task1, task2, task3]);

```

Kod w JavaScript 3: Użycie funkcji **execute()** do zarządzania argumentami *callback*

Kolejnym sposobem na uzyskanie synchronicznego wykonania zadań asynchronicznych jest użycie **Promises**. Jest to podejście bardzo czytelnie i nowoczesne przez co lepsze niż zagnieżdżone callbacks. Funkcja `task()` jest opakowaniem funkcji `printAsync()` w obietnicę **Promise**. Wywołuje funkcję `printAsync()` i po zakończeniu wywołania rozwiązuje obietnicę `resolve(n)` co pozwala kontynuować łańcuch `.then`. Wykorzystanie obietnic przedstawia kod (4).

```
1 function printAsync(s, cb) {
2   var delay = Math.floor((Math.random() * 1000) + 500);
3   setTimeout(function () {
4     console.log(s);
5     if (cb) cb();
6   }, delay);
7 }
8 function task(n) {
9   return new Promise((resolve, reject) => {
10    printAsync(n, function () {
11      resolve(n);
12    });
13  });
14 }
15 task(1).then((n) => {
16   console.log('task', n, 'done');
17   return task(2);
18 }).then((n) => {
19   console.log('task', n, 'done');
20   return task(3);
21 }).then((n) => {
22   console.log('task', n, 'done');
23   console.log('done');
24 });
```

Kod w JavaScript 4: Wykorzystanie mechanizmu obietnic

Następnym podejściem jest użycie konstrukcji *async/await*. Użycie tego mechanizmu poprawia czytelność względem użycia obietnicy w funkcji `task()` z kodu (4), ponieważ funkcja `printAsync()` tworzy obietnicę, którą po zakończeniu działania rozwiązuje. Wywołanie `await printAsync()` powoduje, że kolejne wywołanie zostanie wykonane dopiero po zakończeniu poprzedniego. Kod wykorzystujący konstrukcję *async* oraz *await* zawiera blok (5).

```
1 async function printAsync(s) {
2   var delay = Math.floor((Math.random()*1000)+500);
3   return new Promise((resolve) => {
4     setTimeout(function() {
5       console.log(s);
6       resolve();
7     }, delay);
8   })
9 }
10 async function loop(m) {
11   for (i=0; i<m; i++) {
12     await printAsync("1");
13     await printAsync("2");
14     await printAsync("3");
15   }
16 }
17 loop(4);
```

Kod w JavaScript 5: Wykorzystanie konstrukcji *async* oraz *await*

3 Treść zadań

1. Zaimplementuj funkcję **loop(m)**, która powoduje wykonanie sekwencji zadań z kodu (4) m razy:
 - (a) z wykorzystaniem rekurencji.
 - (b) wykorzystując funkcję **waterfall()** z biblioteki **async**.
2. Proszę napisać program obliczający liczbę linii we wszystkich plikach tekstowych z danego drzewa katalogów. Do testów proszę wykorzystać zbiór danych Traceroute Data. Program powinien wypisywać liczbę linii w każdym pliku, a na końcu ich globalną sumę. Proszę zmierzyć czas wykonania dwóch wersji programu:
 - (a) z synchronicznym (jeden po drugim) przetwarzaniem plików
 - (b) z asynchronicznym (jednocześnie) przetwarzaniem plików

Wykorzystać wzorzec opisany [tutaj](#). Do obliczenia liczby linii w pliku tekstowym wykorzystać kod (11).

```
1 fs.createReadStream(file).on('data', function(chunk) {
2     count += chunk.toString('utf8')
3     .split(/\r\n|[\n\r\u0085\u2028\u2029]/g)
4     .length-1;
5 }).on('end', function() {
6     console.log(file, count);
7 }).on('error', function(err) {
8     console.error(err);
9 });
```

Kod w JavaScript 6: Fragment kodu do użycia w zadaniu 2

4 Realizacja zadania 1a

Implementację funkcji **loop(m)** z wykorzystaniem rekurencji oraz jej wywołanie przedstawia kod (7).

```
1 function loop(m) {
2     if (m <= 0) {
3         console.log('All loops done');
4         return;
5     }
6     task(1)
7     .then((n) => {
8         console.log('task', n, 'done');
9         return task(2);
10    })
11    .then((n) => {
12        console.log('task', n, 'done');
13        return task(3);
14    })
15    .then((n) => {
16        console.log('task', n, 'done');
17        console.log(`Loop ${m} done`);
18        loop(m - 1);
19    });
20 }
21 loop(4);
```

Kod w JavaScript 7: Implementacja funkcji **loop(m)** z wykorzystaniem rekurencji

Funkcja `task(n)` oraz `printAsync(s, cb)` pozostają bez zmian, czyli tak samo jak w kodzie (4). Funkcja `loop(m)` obudowuje sekwencję, którą chcemy wykonać w taki sposób, że jak zakończymy ostatnie zadanie to wywoła się rekurencyjnie z mniejszym licznikiem, który jeśli będzie równy 0 to zakończy działanie. Implementację rekurencyjną łatwo można przerobić na iteracyjną, jednak wykorzystanie rekurencji jest subtelniejsze.

5 Realizacja zadania 1b

W tym zadaniu do implementacji funkcji `loop(m)` musimy wykorzystać funkcję `waterfall()` z biblioteki `async`. Kod tej implementacji zawiera blok (8). W tym przypadku funkcja `printAsync()` pozostaje taka sama jak poprzednio, jednak funkcja `task` musiała ulec zmianie.

```
1 function task(n) {
2   return function (cb) {
3     printAsync(n, function () {
4       console.log('task', n, 'done');
5       cb(null);
6     });
7   };
8 }
9 function loop(m) {
10  if (m <= 0) {
11    console.log('All loops done');
12    return;
13  }
14  async.waterfall(
15    [
16      task(1),
17      (cb) => task(2)(cb),
18      (cb) => task(3)(cb),
19    ],
20    (err) => {
21      if (err) {
22        console.error('Error:', err);
23        return;
24      }
25      console.log(`Loop ${m} done`);
26      loop(m - 1);
27    }
28  );
29 }
30 loop(4);
```

Kod w JavaScript 8: Implementacja funkcji `loop(m)` z wykorzystaniem funkcji `waterfall()`

W powyższym przypadku również mamy do czynienia z rekurencją jednak nad tym, aby zadania były realizowane sekwencyjnie czuwa funkcja `waterfall()`, która otrzymuje listę funkcji jako argument. Każda z tych funkcji otrzymuje dwa parametry, opcjonalny wyniki z poprzedniego korku oraz callback, który należy wywołać, aby przejść do następnego kroku.

6 Realizacja zadania 2

Program do zliczania linii wykorzystuje kilka bibliotek:

- fs - umożliwia operacje na systemie plików
- path - ułatwia pracę ze ścieżkami plików i katalogów
- walkdir - do przeszukania całego drzewa katalogów
- htmlparser2 - parsuje zawartość pliku HTML i tworzy strukturę drzewa DOM (chcemy zliczać tekst, a nie składnię HTML)

Import powyższych bibliotek zawiera blok (9).

```
1 const fs = require('fs');
2 const path = require('path');
3 const walkdir = require('walkdir');
4 const { parseDocument } = require('htmlparser2');
```

Kod w JavaScript 9: Biblioteki potrzebne do zadania 2

Funkcja **extractTextFromHTML(html)** zajmuje się ekstrakcją tekstu z HTML. Parsuje zawartość HTML przy użyciu htmlparser2. Przechodzi rekurencyjnie przez węzły drzewa DOM. Zbiera tekst z węzłów typu text, ignorując tagi HTML. Jej kod zawiera blok (10).

```
1 function extractTextFromHTML(html) {
2   const doc = parseDocument(html);
3   let visibleText = '';
4   function traverse(node) {
5     if (node.type === 'text') {
6       visibleText += node.data.trim() + '\n';
7     } else if (node.children) {
8       node.children.forEach(traverse);
9     }
10  }
11  traverse(doc);
12  return visibleText;
13 }
```

Kod w JavaScript 10: Funkcja wyciągająca tekst z pliku HTML

Funkcja **countVisibleLinesInFile(filePath, callback)** zlicza liczbę widocznych linii tekstu w pliku HTML. Tworzy strumień do odczytu pliku (fs.createReadStream). W trakcie odczytu gromadzi dane w zmiennej html. Po zakończeniu odczytu (end) parsuje zawartość HTML, wyciąga widoczny tekst i zlicza linie. Kod tej funkcji zawiera blok (11).

```

1 function countVisibleLinesInFile(filePath, callback) {
2   let html = '';
3   fs.createReadStream(filePath)
4     .on('data', (chunk) => {
5       html += chunk.toString('utf8');
6     })
7     .on('end', () => {
8       const visibleText = extractTextFromHTML(html);
9       const lines = visibleText.split(/\r\n|[\n\r\u0085\u2028\u2029]/g).filter(
10        (line) => line.trim() !== '');
11       const lineCount = lines.length;
12       console.log(`${filePath}: ${lineCount} visible lines`);
13       callback(null, lineCount);
14     })
15     .on('error', (err) => {
16       console.error(`Error reading file ${filePath}:`, err.message);
17       callback(err);
18     });
19 }

```

Kod w JavaScript 11: Funkcja zliczająca linie

Funkcja **countLinesSync(directoryPath)** synchronicznie zlicza widoczne linie tekstu w plikach HTML. Iteruje po wszystkich plikach HTML w katalogu (przeszukuje drzewo katalogów synchronicznie). Dla każdego pliku:

- Odczytuje zawartość pliku przy użyciu `fs.readFileSync`.
- Wyciąga widoczny tekst za pomocą `extractTextFromHTML`.
- Zlicza widoczne linie i wypisuje wynik.

Sumuje łączną liczbę linii i wypisuje globalny wynik. Jej kod zawiera blok (12).

```

1 function countLinesSync(directoryPath) {
2   const files = walkdir.sync(directoryPath).filter((file) => path.extname(file) === '.html');
3   let totalLines = 0;
4   console.time('Synchronous Execution');
5   files.forEach((file) => {
6     const html = fs.readFileSync(file, 'utf8');
7     const visibleText = extractTextFromHTML(html);
8     const lines = visibleText.split(/\r\n|[\n\r\u0085\u2028\u2029]/g).filter(
9      (line) => line.trim() !== '');
10    const lineCount = lines.length;
11    console.log(`${file}: ${lineCount} visible lines`);
12    totalLines += lineCount;
13  });
14  console.timeEnd('Synchronous Execution');
15  console.log(`Total visible lines (synchronous): ${totalLines}`);
16 }

```

Kod w JavaScript 12: Funkcja przetwarzająca synchronicznie

Funkcja **countLinesAsync(directoryPath)** asynchronicznie zlicza widoczne linie tekstu w plikach HTML. Iteruje po wszystkich plikach HTML w katalogu. Tworzy listę funkcji asynchronicznych do równoległego odczytu i analizy każdego pliku. Zarządza ich równoległym wykonaniem za pomocą funkcji **fork**:

- Po zakończeniu odczytu każdego pliku wywoływany jest callback, który sumuje wynik.
- Kiedy wszystkie pliki zostaną przetworzone, funkcja końcowa wypisuje globalny wynik.

Funkcja **fork(asyncCalls, finalCallback)** synchronizuje asynchroniczne wykonania wielu funkcji. Dla każdej funkcji asynchronicznej tworzy indywidualny callback. Liczy, ile funkcji zostało ukończonych (counter). Gdy wszystkie funkcje zakończą działanie, wywołuje **finalCallback**. Kod obu funkcji zawiera blok (13).

```
1 function countLinesAsync(directoryPath) {
2   const files = walkdir.sync(directoryPath).filter((file) => path.extname(file) === '.html');
3   let totalLines = 0;
4   console.time('Asynchronous Execution');
5   const asyncCalls = files.map((file) => {
6     return (callback) => {
7       countVisibleLinesInFile(file, (err, lineCount) => {
8         if (err) return callback(err);
9         totalLines += lineCount;
10        callback(null);
11      });
12    };
13  });
14  function fork(asyncCalls, finalCallback) {
15    let counter = asyncCalls.length;
16    const checkDone = () => {
17      if (--counter === 0) finalCallback();
18    };
19    asyncCalls.forEach((fn) => fn(checkDone));
20  }
21  fork(asyncCalls, () => {
22    console.timeEnd('Asynchronous Execution');
23    console.log(`Total visible lines (asynchronous): ${totalLines}`);
24  });
25 }
```

Kod w JavaScript 13: Funkcja przetwarzająca asynchronicznie

W celu wykonania obu funkcji w pliku z kodem trzeba umieścić linijki z bloku (14), gdzie **directoryPath** to ścieżka do folderu z plikami do przejrzenia.

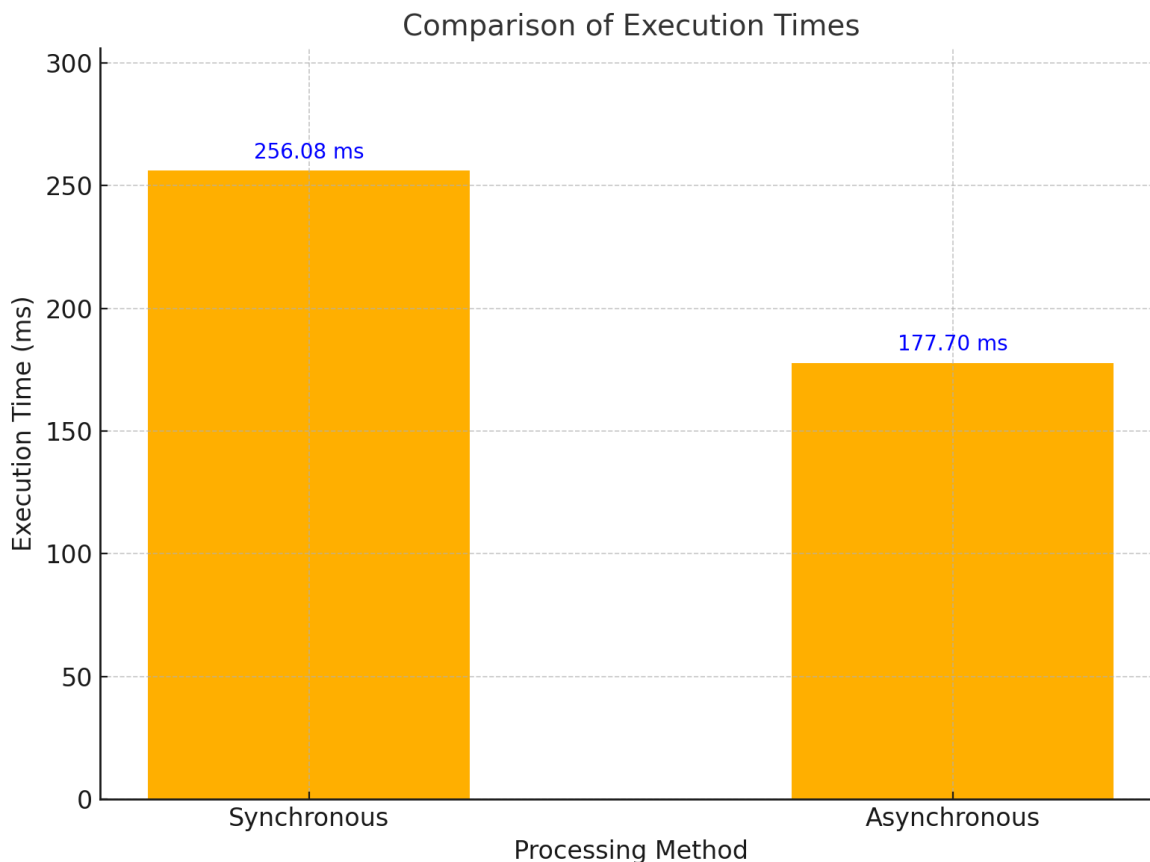
```
1 const directoryPath = './PAM08';
2 countLinesSync(directoryPath);
3 countLinesAsync(directoryPath);
```

Kod w JavaScript 14: Wywołanie funkcji przetwarzających

7 Wnioski i wyniki z zadania 2

Zarówno w trybie synchronicznym, jak i asynchronicznym program zliczył dokładnie tę samą liczbę widocznych linii (45,439). Oznacza to, że oba podejścia przetwarzają dane w sposób spójny i niezawodny. Przetwarzanie asynchroniczne okazało się szybsze od przetwarzania synchronicznego, co jest zgodne z

oczekiwaniami. Asynchroniczne podejście równolegle odczytuje i przetwarza pliki, dzięki czemu efektywniej wykorzystuje zasoby systemowe, minimalizując czas oczekiwania na operacje wejścia-wyjścia (I/O). W tym przypadku asynchroniczna metoda była o około 31% szybsza (177.701 ms vs. 256.084 ms), co stanowi znaczną różnicę, szczególnie w przypadku dużych zestawów danych.



Rysunek 1: Wykres czasu wykonania

W programach, gdzie dominują operacje I/O (np. odczyt wielu plików z dysku), podejście asynchroniczne jest bardziej wydajne, ponieważ unika blokowania procesu podczas oczekiwania na zakończenie operacji. Synchroniczne przetwarzanie może być wystarczające w przypadku mniejszych zestawów danych lub prostych aplikacji, gdzie wydajność nie jest priorytetem.

Literatura

- [1] <https://nodejs.org/api/events.html>
- [2] <https://nodejs.org/api/process.html>
- [3] <https://github.com/creationix/howtonode.org/tree/master/articles>
- [4] <https://kikobeats.com/synchronously-asynchronous>
- [5] <https://levelup.gitconnected.com/javascript-and-asynchronous-magic-bee537edc2da>
- [6] <https://flaviocopes.com/javascript-event-loop/>